

Postgres DBA Basics

Hour 3 of PostgreSQL Training Day

SCaLE LA 2026

Hour 3 Topics

- Postgres hosting options
- Backups: dump/restore, basebackup (hands-on!)
- WAL and incremental backups
- Upgrades and versions
- DR & HA concepts
- Logical replication (hands-on!)
- Connection pooling (hands-on!)
- Disk, storage, and vacuum
- Table partitioning (hand-on!)

Training Materials

github.com/elizabeth-christensen/postgres-full-day-training



Postgres Hosting Options

Where to Run Postgres

Self-Managed

- Bare metal servers
- Virtual machines
- Docker/Kubernetes
- Your laptop

Managed Services

- AWS RDS/Aurora
- Azure Database
- Google Cloud SQL
- Crunchy Bridge / Snowflake
- Neon, Supabase

Self-Managed vs Managed

Aspect	Self-Managed	Managed
Control	Full	Limited
Maintenance	You	Provider
Cost	Lower (maybe)	Predictable
Expertise	Required	Optional
Compliance	Your responsibility	Shared

When to Choose Self-Managed

- Need specific configurations
- Cost optimization at scale
- Regulatory requirements
- Internal expertise available
- Special extensions needed

When to Choose Managed

- Small team / no DBA
- Quick time to market
- Predictable costs important
- Built-in HA/DR needed
- Focus on app development

Backups

Backup Strategy Fundamentals

- **What** to backup: data, configs, WAL
- **Where** to store: local, remote, cloud
- **How often**: RPO (Recovery Point Objective)
- **How fast** to recover: RTO (Recovery Time Objective)

Backup Choices

- Dump/restore - simple, but not automated, long restore times
- Basebackup with added backups - more robust but self managed
- Automated WAL tools - pg_backrest, WAL-E/G - robust, reliable, added complexity

pg_dump - a copy, not a real backup

Note: must have PG 18 installed for this to work locally

Dump entire database (using connection string)

```
pg_dump postgresql://postgres:training@localhost:5432/bluebox > bluebox.sql
```

Dump in custom format (compressed, parallel restore)

```
pg_dump -Fc postgresql://postgres:training@localhost:5432/bluebox > bluebox.dump
```

Dump specific tables

```
pg_dump -t 'bluebox.film' -t 'bluebox.rental' \  
    postgresql://postgres:training@localhost:5432/bluebox > tables.sql
```

pg_dump Options

Option	Description
-Fc	Custom format (recommended)
-Fd	Directory format (parallel)
-j 4	Parallel jobs
--schema-only	Structure only, no data
--data-only	Data only, no structure
-t tablename	Specific table

pg_dumpall - All Databases

Dump all databases including globals (roles, tablespaces)

```
pg_dumpall -d postgresql://postgres:training@localhost:5432/postgres > full_cluster.sql
```

Globals only (roles, tablespaces)

```
pg_dumpall --globals-only -d postgresql://postgres:training@localhost:5432/postgres > globals.sql
```

pg_restore - Restoring Backups

Restore from custom format

```
pg_restore -h localhost -U postgres -d bluebox_new bluebox.dump
```

Parallel restore

```
pg_restore -j 4 -h localhost -U postgres -d bluebox_new bluebox.dump
```

Restore specific table

```
pg_restore -t film -h localhost -U postgres -d bluebox_new bluebox.dump
```

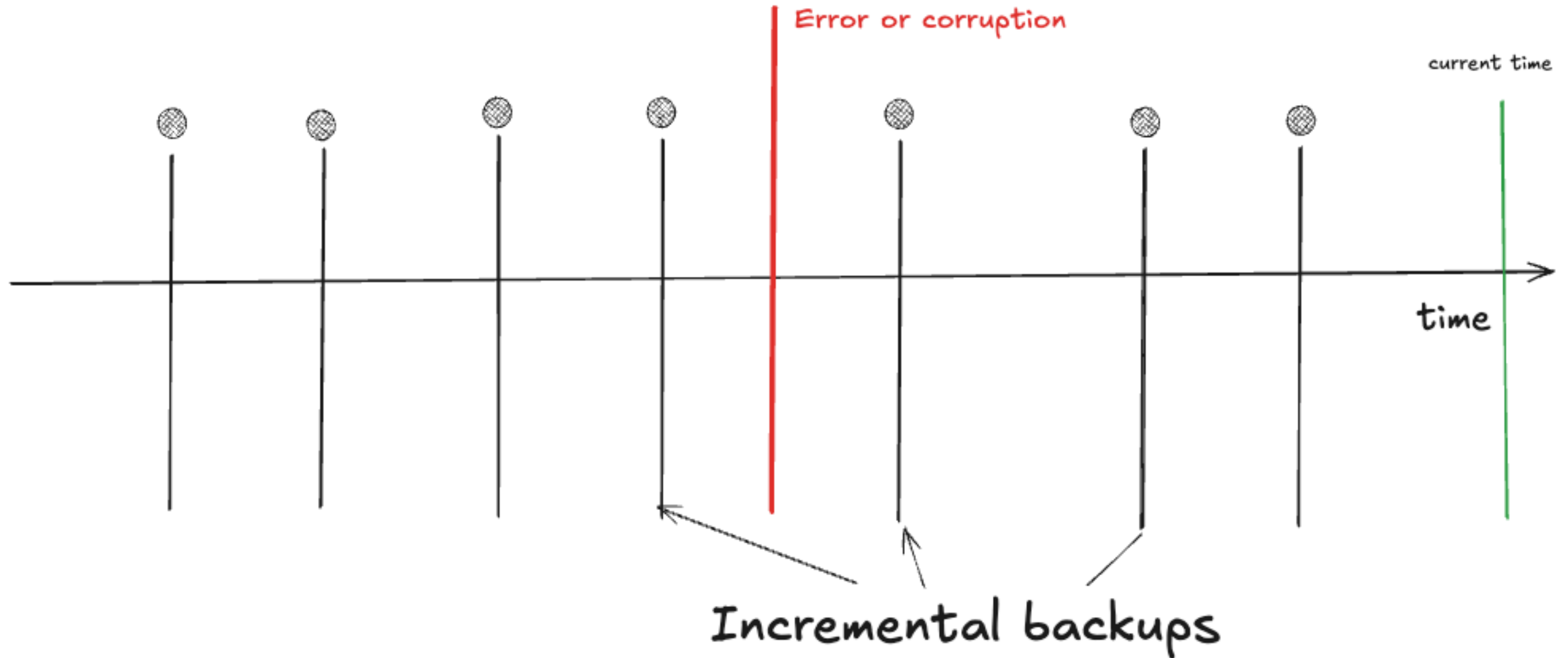
pg_basebackup - Physical Backup

Copies the entire database cluster at the file level.

Unlike pgdump (logical), pgbasebackup:

- Copies raw data files
- Faster for large databases
- Required for streaming replication setup
- Enables point-in-time recovery

Postgres Point in Time Recovery



Hands-On: pg_basebackup

Let's take a real backup in our Docker environment:

Create backup directory inside container

```
docker exec postgres-training mkdir -p /backup
```

Take a full backup (plain format)

Backup goes to /backup/full INSIDE the container

```
docker exec postgres-training pg_basebackup \  
-U postgres \  
-D /backup/full \  
-Fp \  
-Xs \  
-P
```

Note: This backup is inside the container. In production, you'd mount an external volume or copy backups out.

pg_basebackup Options

Option	Description
-D	Destination directory
-Fp	Plain format (directory of files)
-Ft	Tar format (single archive)
-Xs	Stream WAL during backup
-Xf	Fetch WAL after backup
-P	Show progress
-c fast	Fast checkpoint (don't wait)

Inspect the Backup

List backup contents

docker **exec** postgres-training **ls** -la /backup/full/

You'll see the PostgreSQL data directory structure:

base/ - Table data files

global/ - Cluster-wide tables

pg_wal/ - WAL files included in backup

backup_manifest - Backup metadata (v13+)

Verify the Backup Manifest

View the backup manifest (JSON)

```
docker exec postgres-training head -50 /backup/full/backup_manifest
```

```
{ "PostgreSQL-Backup-Manifest-Version": 2,  
  "System-Identifier": "7470012345678901234",  
  "Files": [  
    { "Path": "backup_label", "Size": 225,  
      "Checksum": "abc123..." },  
    ...
```

The manifest lists every file with checksums for verification.

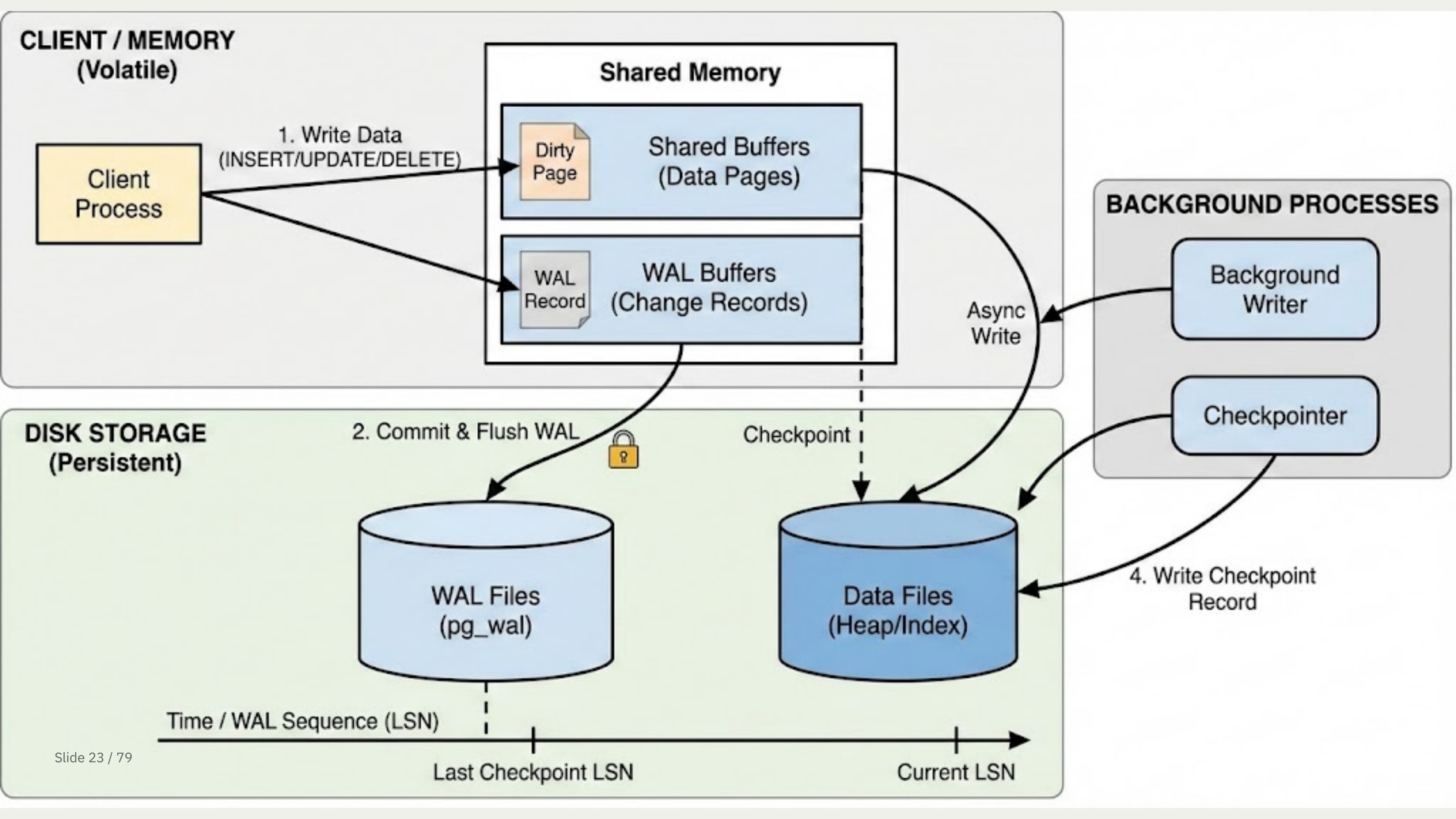
Understanding WAL

Write-Ahead Logging

What is WAL?

Write-Ahead Logging - PostgreSQL's durability mechanism

1. Before data is written to tables, changes are logged to WAL
2. WAL is sequentially written (fast!)
3. On crash, WAL is "replayed" to recover data
4. WAL is also used for replicas, HA scenarios, and robust PITR backup systems





Hands-On: View WAL

-- Current WAL position (Log Sequence Number)

```
SELECT pg_current_wal_lsn();
```

-- 0/3A8B9D0

-- WAL stats

```
SELECT wal_records, wal_bytes,  
       pg_size_pretty(wal_bytes) as wal_size  
FROM pg_stat_wal;
```

List WAL files in the container

```
docker exec postgres-training ls -la \  
/var/lib/postgresql/18/docker/pg_wal/
```

Why WAL Matters for Backups: Consistency

- changes are made to the database during the filesystem copy during pg_basebackup
- without WAL, your copy will have files from before/after changes exist
- no guarantees of consistency; database needs to recover to a known checkpoint
- a data directory copy without WAL generated during the dump IS NOT A BACKUP!

WAL enables [Point-in-Time Recovery \(PITR\)](#)

Base Backup (Monday) + WAL files = Any point in time

- Take a base backup occasionally (daily/weekly)
- Archive WAL files continuously
- Recover to any moment: "Restore to Tuesday 2:47 PM"

Backup Strategy Example

Small database (< 100 GB)

- Daily full backups with pg_dump
- Keep 7 days of backups

Large database (> 100 GB)

- Weekly full pg_basebackup
- Daily incremental backups (v18+)
- Continuous WAL archiving for PITR
- Keep 4 weeks + monthly archives

Physical Backup Summary

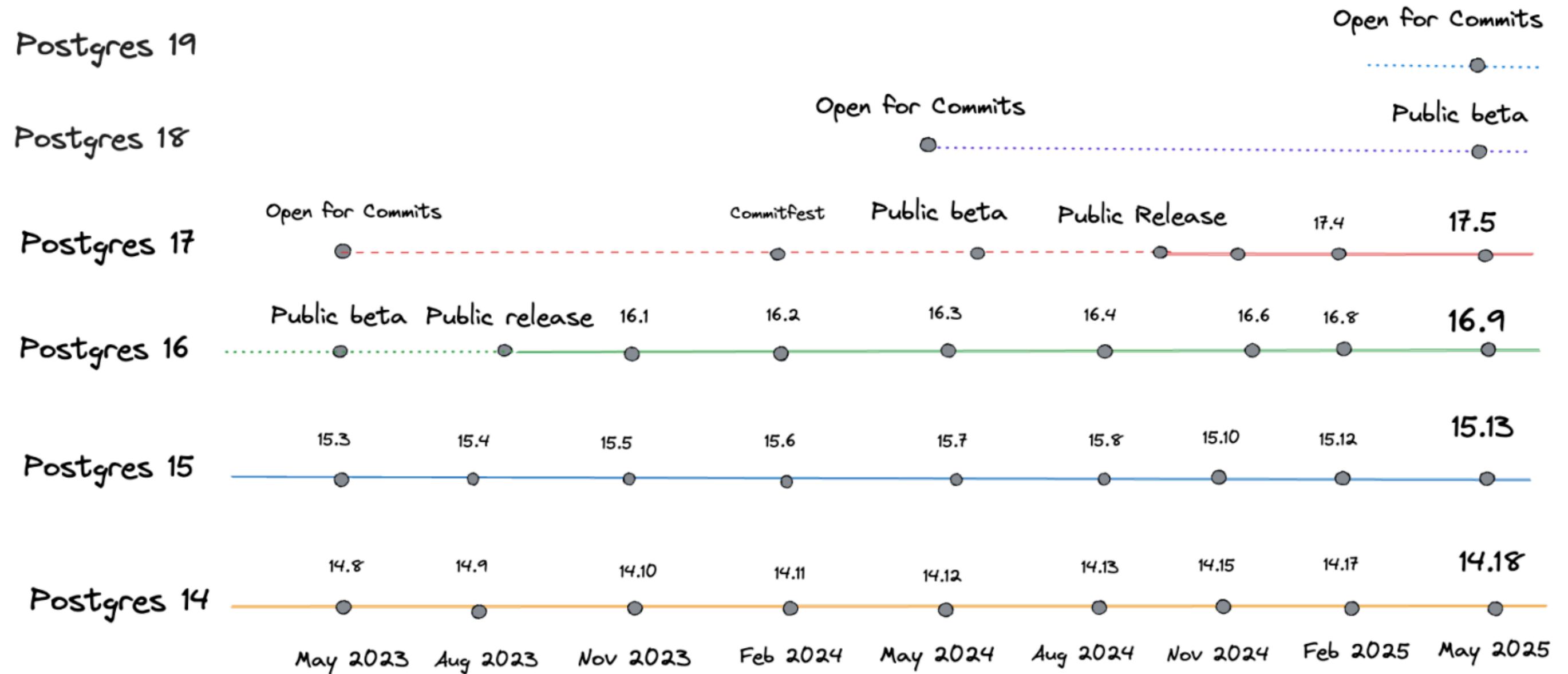
1. `pg_basebackup` - Takes full cluster copy
2. `backup_manifest` - Tracks files and checksums
3. `WAL archiving` - Enables point-in-time recovery
4. `Incremental (v18+)` - Only changed blocks
5. `pg_combinebackup` - Merges incremental chain
6. `pg_verifybackup` - Validates backup integrity

Other Backup Tools

- [pgBackRest](#) - Parallel backup, retention policies, encryption
- [Barman](#) - Backup and recovery manager
- [WAL-G](#) - Cloud-native archival

Upgrades and Versions

Postgres Versions Timeline



Current Support Status

Version	First Release	End of Life
18	Sept 2025	Nov 2030
17	Sept 2024	Nov 2029
16	Sept 2023	Nov 2028
15	Oct 2022	Nov 2027
14	Sept 2021	Nov 2026

Minor Version Upgrades

Simple - just update packages and restart

Ubuntu/Debian

```
apt update && apt upgrade postgresql-18
```

Restart

```
systemctl restart postgresql
```

Always read the release notes!

- may have special instructions for proper upgrade



Minor Upgrade with Docker

Check current version

```
docker exec postgres-training psql -U postgres \  
-c "SELECT version();"
```

Stop containers

```
docker compose down
```

Pull latest image (gets newest 18.x)

```
docker compose pull
```

Start with new image - data volume persists

```
docker compose up -d
```

Verify new version

```
docker exec postgres-training psql -U postgres \  
-c "SELECT version();"
```

Major Version Upgrades - Options

1. [pgdump/pgrestore](#) - Logical, works across versions
2. [pg_upgrade](#) - In-place, faster for large databases
3. [Logical replication](#) - Zero/minimal downtime

pg_upgrade

Stop both clusters

```
systemctl stop postgresql
```

Run pg_upgrade

```
pg_upgrade \  
-b /usr/lib/postgresql/17/bin \  
-B /usr/lib/postgresql/18/bin \  
-d /var/lib/postgresql/17/main \  
-D /var/lib/postgresql/18/main
```

Start new cluster

```
systemctl start postgresql@18-main
```

Pre-Upgrade Checklist

- ☐ Test upgrade in non-production first
- ☐ Review release notes for breaking changes
- ☐ Check extension compatibility
- ☐ Verify disk space (2x database size)
- ☐ Plan maintenance window
- ☐ Script/document all necessary changes
- ☐ Have rollback plan ready

DR & HA Concepts

Key Terms

- **RPO** - Recovery Point Objective (data loss tolerance)
- **RTO** - Recovery Time Objective (downtime tolerance)
- **HA** - High Availability (minimize downtime)
- **DR** - Disaster Recovery (survive catastrophe)

Streaming Replication

Primary server streams WAL to standby

Primary —WAL Stream—> Standby (Hot Standby)

- **Synchronous**: Zero data loss, higher latency
- **Asynchronous**: Some data loss risk, lower latency
- server can be setup as Async for speed, but individual transactions can be sync for protection

Setting Up Streaming Replication

Primary postgresql.conf:

```
wal_level = replica  
max_wal_senders = 3  
wal_keep_size = 1GB
```

Standby postgresql.conf:

```
primary_conninfo = 'host=primary user=replication'  
hot_standby = on # allows queries on the standby
```

Failover Options

Manual

- Promote standby manually
- Update application connections
- Simple but slow

Automatic

- Patroni
- repmgr
- pgpool-II
- Cloud provider HA

Patroni - HA Solution

```
## patroni.yml
```

```
scope: postgres-cluster
```

```
name: node1
```

```
postgresql:
```

```
  listen: "*:5432"
```

```
  data_dir: /data/postgresql
```

```
etcd:
```

```
  hosts: etcd1:2379,etcd2:2379,etcd3:2379
```

Hands-on Patroni is out of scope, but try the official demo:

github.com/patroni/patroni (see docker-compose.yml)

Logical Replication

What is Logical Replication?

Replicates data changes at the logical level (SQL)

Unlike streaming replication:

- Can replicate only certain tables instead of entire cluster
- Can replicate between different versions
- Subscriber can have writes

Logical Replication Use Cases

- Zero-downtime major upgrades
- Consolidating data from multiple sources
- Selective replication (specific rows, types of updates)
- Real-time reporting replicas
- Multi-region distribution
- OS/server architecture differences



Hands-On: Logical Replication

First, enable the DBA profile if you haven't:

```
docker compose --profile dba up -d
```

Container	Port	Role
postgres-training	5432	Publisher
postgres-subscriber	5433	Subscriber

Verify both are running

```
docker ps | grep postgres
```

Step 1: Prepare the Subscriber

Create matching database and table on subscriber:

```
## Connect to subscriber (port 5433)
psql "postgresql://postgres:training@localhost:5433/postgres"
```

```
CREATE DATABASE bluebox;
\c bluebox
CREATE SCHEMA bluebox;
```

```
-- Create custom mpaa_rating type
CREATE TYPE mpaa_rating AS ENUM (
    'G',
    'PG',
    'PG-13',
    'R',
    'NC-17',
    'NR'
);
```

```
[.column]
```

```
-- Create the table structure (no data)
CREATE TABLE bluebox.film (
    film_id bigint primary key,
    title text,
    overview text,
    release_date date,
    genre_ids integer[],
    original_language text,
    rating mpaa_rating,
    popularity real,
    vote_count integer,
    vote_average real,
    budget bigint,
    revenue bigint,
    runtime integer,
    fulltext tsvector
);
```

Step 2: Create Publication

On the publisher (port 5432):

```
psql "postgresql://postgres:training@localhost:5432/bluebox"
```

```
-- Create publication for the film table
```

```
CREATE PUBLICATION film_pub FOR TABLE bluebox.film;
```

```
-- Verify
```

```
SELECT * FROM pg_publication;
```

```
SELECT * FROM pg_publication_tables;
```

Step 3: Create Subscription

On the subscriber (run from inside the container):

```
psql "postgresql://postgres:training@localhost:5433/bluebox"
```

```
-- Subscribe to the publisher (use Docker service name)
```

```
CREATE SUBSCRIPTION film_sub
```

```
CONNECTION 'host=postgres port=5432 dbname=bluebox user=postgres password=training'
```

```
PUBLICATION film_pub;
```

```
-- Check status
```

```
SELECT * FROM pg_stat_subscription;
```

Step 4: Watch It Replicate!

Initial sync happens automatically. Check subscriber:

-- On subscriber (5433)

```
SELECT count(*) FROM bluebox.film;
```

-- Should match publisher!

```
SELECT title, vote_average
```

```
FROM bluebox.film
```

```
ORDER BY vote_average DESC
```

```
LIMIT 5;
```

Step 5: Test Live Replication

On publisher (5432), make a change:

-- Update a film's rating

```
UPDATE bluebox.film
```

```
SET vote_average = 9.9
```

```
WHERE title = 'The Dark Knight';
```

On subscriber (5433), see the change:

```
SELECT title, vote_average
```

```
FROM bluebox.film
```

```
WHERE title = 'The Dark Knight';
```

-- vote_average is now 9.9!

Monitor Replication Status

-- On subscriber: check replication lag

SELECT

subname,

received_lsn,

latest_end_lsn,

last_msg_receipt_time

FROM pg_stat_subscription;

-- On publisher: see active replication slots

SELECT slot_name, active, restart_lsn

FROM pg_replication_slots;

Clean Up (Optional)

-- On subscriber: drop subscription

```
DROP SUBSCRIPTION film_sub;
```

-- On publisher: drop publication

```
DROP PUBLICATION film_pub;
```

Connection Management

Connection Settings

-- postgresql.conf

max_connections = 100 -- Maximum concurrent connections

superuser_reserved_connections = 3

-- View current connections

```
SELECT count(*) FROM pg_stat_activity;
```

-- View connection details

```
SELECT username, application_name, client_addr, state
```

```
FROM pg_stat_activity
```

```
WHERE datname = 'bluebox';
```

The Connection Problem

Each Postgres connection = 1 process

- Memory overhead (~5-10MB per connection)
- Context switching costs
- Applications leave idle connections
- `max_connections` has practical limits

See idle connections:

```
SELECT pid, username, state, query_start, state_change
FROM pg_stat_activity
WHERE state = 'idle'
ORDER BY state_change;
```

Solution: Connection pooling

PgBouncer - Connection Pooler

PgBouncer sits between your app and PostgreSQL:

App (1000 connections) → PgBouncer → PostgreSQL (20 connections)

- Lightweight (single process, low memory)
- Reuses database connections across clients
- Your Docker Compose already has it running!

Hands-On: PgBouncer

First, enable the DBA profile if you haven't:

```
docker compose --profile dba up -d
```

Verify PgBouncer is running

```
docker ps | grep pgbouncer
```

Should show: pgbouncer-training on port 6432

Pool Modes

Mode	Description	Use Case
session	One server conn per client	Legacy apps
transaction	Shared between transactions	Most apps
statement	Shared between statements	Simple queries

Our Docker setup uses **transaction** mode (most common).

Connect Through PgBouncer

Direct to PostgreSQL (port 5432)

```
psql "postgresql://postgres:training@localhost:5432/bluebox"
```

Through PgBouncer (port 6432)

```
psql "postgresql://postgres:training@localhost:6432/bluebox"
```

Both connections work the same - but PgBouncer pools them!

PgBouncer Admin Console

PgBouncer has a special admin database for monitoring:

Connect to PgBouncer admin (use pgbouncer as database name)

```
psql "postgresql://postgres:training@localhost:6432/pgbouncer"
```

-- Show all available commands

```
SHOW HELP;
```

Monitor Pool Statistics

-- From PgBouncer admin console

SHOW POOLS;

database	user	cl_active	cl_waiting	sv_active	sv_idle
bluebox	postgres	1	0	1	4

- cl_active: Client connections actively using a server connection
- sv_idle: Server connections waiting in pool

Disk, Storage, and Vacuum

MVCC - Multi-Version Concurrency Control

PostgreSQL keeps old row versions for:

- Read consistency
- Transaction rollback
- No read locks on writes

But: Dead rows accumulate!

What is Vacuum?

VACUUM reclaims space from dead rows

-- Manual vacuum

VACUUM bluebox.rental;

-- Vacuum with analysis

VACUUM ANALYZE bluebox.rental;

-- Full vacuum (rewrites table, locks!)

VACUUM **FULL** bluebox.rental;

Autovacuum

Postgres automatically vacuums tables

-- Key settings

autovacuum = on

autovacuum_vacuum_threshold = 50

autovacuum_vacuum_scale_factor = 0.2

-- 20% of rows changed + 50 = trigger vacuum

Monitoring Table Bloat

-- Check dead tuples

SELECT

schemaname,

relname,

n_dead_tup,

n_live_tup,

round(n_dead_tup::numeric / NULLIF(n_live_tup, 0) * 100, 2) as dead_pct

FROM pg_stat_user_tables

WHERE n_dead_tup > 0

ORDER BY n_dead_tup DESC

LIMIT 10;

Table and Index Bloat

-- pg_bloat_check or pgstattuple extension

```
CREATE EXTENSION pgstattuple;
```

```
SELECT * FROM pgstattuple('bluebox.rental');
```

Tools for bloat analysis:

- check_postgres (Nagios plugin)
- pgstattuple extension
- Various community queries

Disk Space Monitoring

```
SELECT pg_size_pretty(pg_database_size('bluebox'));
```

```
-- Result: 876 MB
```

```
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) as size
FROM pg_stat_user_tables WHERE schemaname = 'bluebox'
ORDER BY pg_total_relation_size(relid) DESC LIMIT 5;
```

relname	size
rental	286 MB
payment	181 MB
inventory	162 MB
person	54 MB
film_crew	45 MB

Table Partitioning

What is Partitioning?

Breaking a large table into smaller physical pieces

payment (one huge table)



payment_2024 | payment_2025 | payment_2026
(small) | (small) | (small)

PostgreSQL automatically routes queries/inserts to the right partition.

Why Partition?

- **Query performance:** Scans only relevant partitions (partition pruning)
- **Data management:** DROP old partitions instead of DELETE
- **Maintenance:** VACUUM/REINDEX smaller chunks
- **Parallel queries:** Each partition can be scanned in parallel

Best for: Large tables (100M+ rows), time-series data, archival needs

Partition Types

Type	Use Case	Example
RANGE	Time-series, sequential	Payments by month
LIST	Categories, regions	Orders by country
HASH	Even distribution	Users by ID



Hands-On: Range Partitioning

-- Create partitioned table

```
CREATE TABLE bluebox.payment_history (  
  payment_id SERIAL,  
  payment_date TIMESTAMPTZ NOT NULL,  
  customer_id INT,  
  amount NUMERIC(5,2)  
) PARTITION BY RANGE (payment_date);
```

-- Create partitions for each year

```
CREATE TABLE payment_2024 PARTITION OF bluebox.payment_history  
  FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

```
CREATE TABLE payment_2025 PARTITION OF bluebox.payment_history  
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Insert & Query Partitioned Tables

-- Insert goes to correct partition automatically

```
INSERT INTO bluebox.payment_history (payment_date, customer_id, amount)
VALUES ('2025-06-15', 12345, 4.99);
```

-- Query the parent - PostgreSQL prunes partitions

```
EXPLAIN SELECT * FROM bluebox.payment_history
WHERE payment_date >= '2025-01-01';
```

-- Shows: Scans only payment_2025!

Managing Partitions

-- Add a new partition (before data arrives!)

```
CREATE TABLE payment_2026 PARTITION OF bluebox.payment_history  
    FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');
```

-- Archive old data: detach and drop

```
ALTER TABLE bluebox.payment_history  
    DETACH PARTITION payment_2024;
```

```
DROP TABLE payment_2024; -- or archive to cold storage
```

Partitioning Tips

- Add partitions **before** you need them (or use DEFAULT)
- Partition key must be in PRIMARY KEY
- Foreign keys to partitioned tables: use the parent
- Consider monthly vs yearly based on data volume
- Use pg_partman extension for automated management

Hour 3 Summary

-  Hosting options: self-managed vs managed
-  Backup strategies: pg_dump, basebackup, tools
-  WAL: durability, archiving, PITR
-  Incremental backups: full vs incremental vs differential
-  Version upgrades: minor and major
-  HA/DR: streaming replication, failover
-  Logical replication (hands-on!)
-  Connection pooling with PgBouncer (hands-on!)
-  VACUUM and storage management
-  Table partitioning: RANGE, LIST, HASH

Questions?

Next: Postgres Troubleshooting